WORKSHOP DÉVELOPPEURS

Continuer l’aventure Clever Cloud

Explorer la plateforme et son écosystème, en direct…
et parler de vos projets

Horacio Gonzalez · 27 octobre 2026 · 2 heures en direct · lab à refaire après : github.com/CleverCloud/workshop-continuer-aventure-2026-10-27

PROGRAMME

Deux heures : vous suivez ou vous regardez

Heure	Séquence	Format
14:00	Accueil · création de compte	Sondage · Étape 0
14:10	Le modèle mental : qui fait quoi	Exposé
14:20	Déployer une app Python	Démo · Partie 1
14:45	Add-ons : une base PostgreSQL	Démo · Partie 2
15:05	Quand ça casse : deux pannes classiques	À vous de trouver · Partie 3
15:25	Une app moderne : Django + uv	Démo · Partie 4
15:35	Aller plus loin, dont Kubernetes	Démo · Partie 5
15:45	Vos projets · crédits	Questions-réponses

LE MODÈLE MENTAL

Vous gardez le code. La plateforme fait le reste.

Savoir où passe la frontière, c’est savoir qui doit corriger quoi.

QUI FAIT QUOI

Ce qui reste à vous, ce que la plateforme prend

Couche	Vous	La plateforme
Code, dépendances	✔ git push	—
Configuration	✔ variables d’env.	injecte CC_* et add-ons
Build, démarrage, health check	—	✔
OS, runtime, correctifs	—	✔
Load balancing, TLS	vous choisissez les domaines	✔
Scalabilité	vous fixez les bornes	✔
Données	✔ vos données	bases managées (add-ons)

RESSOURCES

Tout vit dans une organisation

Organisation

membres · rôles · facturation et crédits

Applications

code + runtime + variables d'environnement + scalabilité

Add-ons

PostgreSQL, Redis, Cellar… liés aux applications

Clusters Kubernetes

CKE, en bêta : quand vous voulez Kubernetes

CYCLE DE VIE D'UN DÉPLOIEMENT

Chaque déploiement crée de nouvelles instances

Push
git push ou clever deploy

→

Build
détection du runtime, dépendances, hooks

→

Démarrage
nouvelles instances, variables des add-ons injectées

→

Contrôle
port 8080 ouvert, health check OK

→

Bascule
le trafic bascule, les anciennes instances s'arrêtent

Le contrôle échoue ? Le déploiement échoue et la version précédente continue de servir.

Cache de build : un restart réutilise le dernier build ; un rebuild repart de zéro.

QUATRE RÈGLES DU JEU

Ce que la plateforme attend de votre application

Le code dans Git

Un commit = un déploiement immuable. Rollback = redéployer un commit plus ancien.

La config en variables

Les vôtres, celles de la plateforme (CC_*), celles des add-ons. Modification → restart.

L’état dans les add-ons

Le disque de l’instance est éphémère : les données vont dans une base ou un stockage objet.

Écouter sur 0.0.0.0:8080

Les logs vont sur stdout ; la plateforme les collecte.

PARTIE 1 · 25 MIN

Python : trois fichiers, un réglage

app.py — une petite application Flask
requirements.txt — Flask, psycopg
.gitignore

Ni configuration serveur, ni Dockerfile, ni script de build. La plateforme détecte Python et sert l'application avec uWSGI derrière Nginx.

Seul réglage obligatoire :
CC_PYTHON_MODULE=app:app

```
cp -r $LAB/demos/1-app-python \
~/cc-workshop-<prenom>
git init && git add . \
&& git commit -m "v1"

clever create --type python \
--region par app-<prenom>
clever env set CC_PYTHON_MODULE app:app
clever env set CC_PYTHON_VERSION 3.13
clever deploy
```

Le log raconte l’histoire — lisez-le d’abord

Repérez la ligne de chaque étape :

- le commit déployé
- la version de Python choisie
- l’installation des dépendances
- le cache de build
- le contrôle du port 8080
- le succès du déploiement

```
clever domain
clever open
clever activity
clever logs --since 10m
clever accesslogs --since 10m

curl https://<domaine-app>/
```

Blue/green : la nouvelle version démarre avant que l'ancienne s'arrête




```
while true; do curl -s https://<domaine-app>/ | grep -o '"version":"[^"]*"'; sleep 1; done
```

PARTIE 2 · 15 MIN

Add-ons : des services managés à côté de vos apps

Bases de données, caches, stockage objet, systèmes de fichiers, identité, analytics… Chaque add-on a des **plans** (taille, limites) et des **zones**.

La plateforme les provisionne, les supervise et les sauvegarde. Vous choisissez le plan et possédez les données.

Avant de lier : **curl …/db** → que répond l'app ?

```
clever addon providers
clever addon providers \
show postgresql-addon

clever addon create \
postgresql-addon pg-<prenom> \
--plan dev --region par
clever service link-addon pg-<prenom>
```

Lier injecte les variables de l’add-on au démarrage de l’app



Une instance déjà démarrée ne les voit pas → clever restart. La question n°1 : « mon app ne voit pas la base ».

PARTIE 3 · 20 MIN

Quand ça casse

Un collègue vous envoie sa nouvelle version. Vous déployez.
Le message arrive. À vous de trouver — dans le chat.

Partez du symptôme et des logs, pas du diff : dans la vraie vie, on a rarement le diff.

« Ma nouvelle version n’est pas en ligne »

« J’ai poussé la v2 il y a une heure et l’app affiche toujours la v1 ! La plateforme est en panne ? »

1. La plateforme est-elle en panne ? Et l’app ?
2. Où est l’explication ?
3. Pourquoi l’app répond-elle encore ?
4. Comment corriger ?

```
curl https://<domaine-app>/
clever activity

# Console : Activity →
# le déploiement → ses logs
```

« Ça marche, puis ça plante »

« /db marche quelques fois, puis que des erreurs. Un redémarrage règle le problème… pendant une minute. La base est cassée ? »

1. Le déploiement a-t-il réussi ?
2. Où est l'erreur ? Que dit-elle, jusqu'au bout ?
3. Qui parle : l'app ou la base ?
4. Quelles sont les limites du plan ?
5. Pourquoi le restart « répare » ?

```
for i in $(seq 1 10); do
curl -s -o /dev/null \
-w "%{http_code}\n" \
https://<domaine-app>/db
done
clever logs --since 10m
```

C’est le projet qui décide comment Python tourne

	Mode classique	Mode uv natif
Déclenché par	requirements.txt	uv.lock + CC_PYTHON_UV_RUN_COMMAND
Dépendances	pip install	uv sync
Serveur HTTP	Nginx + uWSGI	le vôtre (uvicorn…)
Point d’entrée	CC_PYTHON_MODULE	CC_PYTHON_UV_RUN_COMMAND
Port	géré par la plateforme	votre serveur sur 0.0.0.0:8080

Au chat : d’où viennent les identifiants de la base ? Migrations dans CC_PRE_RUN_HOOK avec 3 instances ? « Bad Request (400) » ?

PARTIE 5 · ALLER PLUS LOIN

La même app, en production

Scalabilité

Taille et nombre d’instances, auto-scaling horizontal et vertical, dans les bornes que vous fixez.

Votre domaine

clever domain add, certificats TLS automatiques.

Votre stack

Node.js, Java, PHP, Go, Rust, Ruby, .NET, Docker… même modèle, autres réglages.

Votre CI/CD

Déploiement depuis GitHub, ou clever deploy dans votre pipeline.

PARTIE 5 · DÉMO

Et quand vous voulez Kubernetes ?

Clever Kubernetes Engine : Clever Cloud opère le control plane
et remplace les VM en panne. Vous déployez avec kubectl.

```
clever features enable k8s
clever k8s create demo --watch
clever k8s get-kubeconfig demo > ~/.kube/cc-demo.yaml
```

En bêta · activé par utilisateur · en ligne de commande pour l'instant

À RETENIR

Six réflexes pour vos prochains déploiements

1. Vous gardez le code, la configuration et les données ; la plateforme fait le reste.
2. Chaque déploiement crée de nouvelles instances ; leur disque ne survit pas.
3. La configuration, ce sont des variables : les vôtres, CC_*, celles des add-ons. Modification → restart.
4. Un build raté ne casse pas la prod : la version précédente continue de servir.
5. Lisez le log, et l’erreur jusqu’au bout.
6. Un restart n’est pas un correctif.

À VOUS · 15 MIN

Parlons de vos projets

Vos crédits

Le code est dans le chat, maintenant.

Pensez à supprimer les apps du workshop (Partie 6 du lab) pour les garder pour vos projets.

Pour refaire et aller plus loin

Le lab et les démos :
github.com/CleverCloud/
workshop-continuer-aventure-2026-10-27
La doc : clever.cloud/developers/doc
Le CLI : clever.cloud/developers/doc/cli-reference

Votre stack, votre base existante, votre CI, vos contraintes : **posez vos questions**.